Чек-лист
по API проверкам

Nadezhda Dudnik

April, 2023



Приветствие от Надежды

Меня зовут Надежда Дудник, я автор этого чеклиста.

Я главный инженер по тестированию ПО и ментор по тестированию ПО

Веду блог про тестирование ПО, также обучаю и направляю людей, провожу консультации по тестированию ПО.

Мне нравится передавать свои знания, сочинять тесты для закрепления знаний, делиться свои опытом.

Данный чек-лист поможет вам:

- сгенерировать еще больше тестовых идей при тестировании продукта;

- различать негативные и позитивные проверки API;

- структурировать свои знания;

- закрепить свои знания;

- при решении и выполнении тестовых и проектных задач;

- узнать особенности тестирования API.

Чек-лист в виде mindmap

https://mm.tt/map/2696738039?t=T4hTfoUkAt



Чек-лист в виде таблицы - позитивные проверки

№	Проверки (Что тестируем?)	Статус прохождения	Ссылка на дефект	Комментарий
https://t.me/protestinginfo	отправить данные с			
	➖Отправить #GET с обязательными валидными параметрами или валидным телом в запросе (#POST, #PUT, #DELETE, #PATCH);			
	➖Самое главное проверяем ответ от сервера согласно требованиям;			
	➖проверить версию протокола;			
	➖проверить код состояния согласно ТЗ, например, что запрос должен возвращать код состояния HTTP 2XX для GET, POST, DELETE, PUT;			
	➖проверить пояснение (status text);			
	➖проверить сформированный объект JSON (либо XML), а именно структуру ответа согласно модели данных из требований;			
	➖проверить объект в формате JSON с запрошенными полями либо проверить структуру XML ответа от сервера согласно требованиям;			
	➖проверить имена свойств (ключей), типы полей, значения полей согласно требованиям;			
	➖проверить, что у обязательных полей значения не NULL в полученном ответе, опираясь на требование ;			
	➖проверить состояние после отправки запросов, например, для запросов GET убедитесь, что система не изменено состояние, а именно проверить идемпотентность;			
	➖проверить идемпотентность для методов PUT и DELETE;			
	➖проверить безопасность и кэшируемость методов;			

	➖проверить основные заголовки HTTP headers, заголовки ответа согласно требованиям;			
	➖проверить, что информация не утекает через заголовки (например, X-Powered-By заголовок не отправляется пользователю);			
	➖проверить структуру URL согласно требованиям;			
	➖проверить создание токена авторизации;			
	➖проверить структуру полезных данных (payload) токена, который представлен закодированной строкой;			
	➖проверить различные типы токенов авторизации согласно ТЗ;			
	➖проверить создание id или uuid			
	➖проверить обновление существующих данных;			
	➖проверить удаление существующего объекта;			
	➖проверить, что ответ на запрос приходит своевременно			
	➖проверить, что токен авторизации действительный;			
	➖проверить, параметры при отправке запроса;			
	➖проверить, валидное тело запроса при отправке на сервер			

Чек-лист в виде таблицы - негативные проверки

№	Проверки (Что тестируем?)	Статус прохождения	Ссылка на дефект	Комментарий
https://t.me/protestinginfo	отправить данные с			Ожидаемый результат при негативном тестировании это должны быть корректный статус, корректное сообщение, и корректное тело ответа от сервера согласно ТЗ
Общие проверки				
Обязательность полей				
	обязательными полями, не допускающие значения null, являются пустыми			Проверка обязательности полей
	пустыми значениями во вложенных объектах в формате JSON, где поля являются обязательными			
	пустыми значениями во вложенных массивах в формате JSON, где поля являются обязательными			
	без указания значения обязательного поля			
	с дубликатами значений обязательных полей в теле запроса			
	пустыми обязательными HTTP заголовками			
Невалидные, несуществующие и пустые значения				
	невалидным токеном			
	пустым токеном			
	несуществующим id			

	невалидным id			
	невалидным uuid в параметрах запроса			
	невалидным uuid в теле запроса			
	недопустимым uuid			
	несуществующим uuid			
	невалидными HTTP заголовками			
	неподдерживаемые методами			
	невалидным телом в запросе			
	невалидным типом параметра			
	недопустимыми символами в параметрах			
	валидными значениями параметров			
	недопустимыми символами в теле запроса			
JSON формат				
	недопустимыми данными JSON в теле запроса			
	неполной JSON моделью с отсутствующими полями			
	JSON моделью с невалидными полями			
	полностью пустым телом формата JSON			
Переполнением данных				
	uuid длиной порядка 500 символов или больше			
	id длиной порядка 100 символов			
	переполнением тела ответа запроса при отправке (большой JSON)			
Другие картинки				
	с комментариями в теле запроса			
	регистрозависимость в теле запроса			
	регистрозависимость в заголовках			

	регистрозависимость текстовых параметров			
	загрузка с невалидным форматом файла через form-data			
	отправка фото в сломанном формате base64 через Postman			
Проверка на 400 Http Status code				
	400: Неверный запрос. Это может быть вызвано различными действиями пользователя			
	401: Аутентификация не прошла.			
	403: Аутентифицированному пользователю не разрешен доступ к указанной конечной точке API.			
	404: Запрашиваемый ресурс не существует.			
	405: Метод не разрешен. Проверьте заголовок Allow на предмет разрешенных методов HTTP.			
	410: Запрашиваемый контент удалён с сервера.			
	412: Указаны условия в заголовках, которые сервер не может выполнить			
	415: Неподдерживаемый тип медиа. Запрошенный тип содержимого или номер версии недопустимы			
	422: Не удалось выполнить проверку данных (например, в ответ на POST-запрос).			
	429: Слишком много запросов. Запрос был отклонен из-за ограничения скорости.			
Проверка на 500 Http Status code				
	500: Внутренняя ошибка сервера. Это может быть вызвано внутренними ошибками программы.			Смоделировать и сломать то, что не покрыто в коде
HTTP методы (материал [1] от подписчика)				
Авторизация с использованием методов аутентификации [2] [3] [4] [5]				

	Без «токена»			
	С «токеном» для 403 ошибки			
	С невалидным «токеном»			
GET запросы[6] [7]				
	Пустой список (если возможно)			
	Заполненный список			
	Пагинация в списке (limit, offset)[8]			
	Получение списка с ограничением на количество записей			
	Получение списка начиная с указанного номера			
	При отрицательном offset список пользователей возвращается начиная с первой позиции			
	В случае передачи параметров с некорректным значением возвращается 400 с описанием ошибки в теле ответа			
	При несуществующем offset возвращается пустой список			
	Запрос данных по валидному ID, проверка возвращения корректных данных			
	Запрос данных по несуществующему ID, но в валидном формате			
	Запрос данных по невалидному ID			
POST запросы[9] [10] [11]				
	Заполнены все поля валидными данными			
	Заполнены только обязательные поля			
	Заполнены не все обязательные поля			
	Не заполнено ни одно поле			
	Валидация данных в полях (невалидные данные)			
	Пустой JSON			
	Дата создания объекта			
PUT запросы[12]				

	Обновление с невалидными данными			
	Обновление по несуществующему ID			
	Обновление по невалидному ID			
	Валидация полей (невалидные данные)			
	Частичное обновление (в JSON присутствуют не все поля)			
DELETE запросы[13] [14]				
	Удаление несуществующего объекта			
	Удаление уже удаленного объекта			
	Удаление по несуществующему ID			
	Удаление по невалидному ID			
	Удаление и повторное добавление такой же сущности (если есть уникальные поля)			
Другое				
	Протокол HTTP / HTTPS согласно спецификации: например, Mixed Content: The page at 'https://<>.ru/' was loaded over HTTPS, but requested an insecure stylesheet 'http://fonts.googleapis.com/css2?family=Arimo:wght@400;700&family=Play:wght@400;700&display=swap'. This request has been blocked; the content must be served over HTTPS.			
	Ошибочные форматы обмена данными (json, xml, html)			
	Неверная полезная нагрузка ответа			
	Замена допустимого метода на недопустимый метод			
	Большое или недопустимое Время отклика			
	Наличие / отсутствие файла (-ов) куки			

Моя статья на Habr

Шаблоны тест-кейсов по API, тест-кейсы по идемпотентности

https://habr.com/ru/articles/718560/

Полезные ссылки от моего подписчика

Полезный материал: Чек-лист API проверок
[1] Postman чек-лист | Postman тестирование api
[2] Authorizing requests
[3] How to write authentication test cases with Postman?
[4] Аутентификация в веб-API: пример и ликбез по Postman
[5] ТЕСТИРОВАНИЕ АВТОРИЗАЦИИ OAUTH2 В POSTMAN
[6] Тестирование GET-запросов
[7] Как пользоваться Postman 2_Выполнение_запроса
[8] Постраничная выборка (limit и offset)
[9] Тестирование POST-запросов
[10] Выполнение запроса
[11] All types of POST Requests with Postman
[12] API для QA: учимся тестированию ПО без доступа к коду
[13] Postman - DELETE Requests
[14] Postman Tutorial #9 - How to Send DELETE Request in Postman
[15] Как установить Cookie в Postman. Тестирование Cookie в Postman

Задание

Коллекция в swagger, с которой предстоит работать:
https://petstore.swagger.io/

Предусловие:
- Base URL: https://petstore.swagger.io/
(https://petstore.swagger.io/)- Request URL: https://petstore.swagger.io/v2/pet
- Headers: 'content-type': application/json
- "status" питомца в теле запроса: "available"

Задание: необходимо создать коллекцию в Postman
Запросы, которые должны быть в коллекции:

1. Создать нового питомца (метод POST /pet), заполнить все ключи своими примерами значений.
Прописать скрипт на вкладке tests, который берет из тела ответа id созданного питомца и записывает в коллекцию.
2. Прописать скрипт, который берет из тела ответа name (название) tags питомца и записывает в коллекцию. А также указать проверку на pm.test , что название tags из тела запроса равно названию tags из тела ответа сервера.
3. Получить информацию питомца по id (метод GET /pet/{petId}), используя в запросе ранее записанную переменную id pet из коллекции.
Тут же на вкладке tests прописать скрипт, который будет отправлять следующий запрос "Updates a pet in the store with form data" (метод POST /pet/{petId}) на обновление ключа «status» на «sold» (продан).
Подсказка: использовать pm.sendRequest
4. То, что касается негативных проверок - создать дополнительные запросы:
- Id питомца невалиден при создании - ошибка 400 (bad input)
- Id питомца не существует при получении - ошибка 404 (Pet not found)
- неподдерживаемый формат данных при создании - 415 ошибка Unsupported Media Type
- при создании питомца, что нужно сделать, чтобы отобразилась такая ошибка 500 Server Error?

```
{
   "code": 500,
   "type": "unknown",
   "message": "something bad happened"
}
```

Коллекцию необходимо экспортировать (Export/ Export Collection) и отправить на проверку.
Если хотите еще напишите тест-кейсы по API😄

Контакты

ДО ВСТРЕЧИ В МОЕМ БЛОГЕ!

в моем блоге / телеграм канале или переходите на мой сайт Protestinginfo